

UNIVERSITÉ DE NOUAKCHOTT
Faculté des Sciences et Techniques

Master 1 Intelligence Artificielle

Méthodes d'Ensemble & Fondements des Réseaux Profonds

Advanced Machine Learning (AML) — Advanced Deep Learning (ADL)

Présenté par :

Mohamed Salem Ebnou Oubeid — C34613

Fatimata Issa Saw — C21304

Oussama Sid'Ahmed Hedy — C34603

Encadreur : Dr. Rivea Sadegh

Date de soutenance : 13 juin 2026

Rapport AML & ADL — Paire 1

1. Introduction

Ce rapport présente les travaux réalisés dans le cadre du module Advanced Machine Learning (AML) et Advanced Deep Learning (ADL) — Paire 1. Il couvre deux axes complémentaires : les méthodes d'ensemble en Machine Learning classique, et les fondements mathématiques des réseaux de neurones profonds.

Ces deux domaines partagent un lien fondamental : la technique du Dropout, omniprésente en Deep Learning, peut être interprétée comme une forme implicite de Bagging. Ce rapport explore ce pont conceptuel et présente les implémentations associées.

2. Méthodes d'Ensemble (AML)

2.1 Décomposition Biais-Variance

L'erreur quadratique moyenne d'un estimateur se décompose en trois termes fondamentaux :

$$\mathbb{E}[(y - \hat{y})^2] = \text{Biais}^2 + \text{Variance} + \sigma^2$$

Le biais représente l'erreur systématique due à un modèle trop simple (sous-apprentissage). La variance mesure la sensibilité du modèle aux fluctuations des données d'entraînement (sur-apprentissage). Le bruit irréductible σ^2 est inhérent aux données. Le Bagging réduit la variance ; le Boosting réduit le biais.

2.2 Théorème de Condorcet

Ce théorème garantit que le vote majoritaire de N classifieurs indépendants, chacun correct avec probabilité $p > 0,5$, est supérieur à un classifieur unique. La probabilité d'une décision correcte vaut :

$$P(\text{majorite correcte}) = \sum_{k=\lfloor N/2 \rfloor + 1}^N \binom{N}{k} p^k (1-p)^{N-k}$$

À mesure que $N \rightarrow \infty$, cette probabilité tend vers 1 (tant que $p > 0,5$). Notre simulation Monte Carlo avec 50 000 itérations confirme ce résultat théorique à $\pm 0,001$ près.

2.3 Bagging, Boosting et Stacking

Méthode	Principe	Réduit	Parallélisable
Bagging	Bootstrap + agrégation	Variance	Oui
Random Forest	Bagging + features aléatoires	Variance	Oui
AdaBoost	Poids adaptatifs séquentiels	Biais	Non
Grad. Boosting	Descente de gradient séquentielle	Biais	Non
XGBoost	GB + régularisation L1/L2	Biais	Partiel
Stacking	Méta-apprenant sur prédictions OOF	Biais + Var.	Oui (niv. 1)

Nos expériences sur le dataset Digits (1 797 images, 10 classes) montrent que Random Forest (100 arbres) atteint 0,972 d'accuracy contre 0,873 pour un arbre seul. XGBoost atteint 0,979 avec un temps d'entraînement compétitif.

3. Fondements des Réseaux Profonds (ADL)

3.1 Du Perceptron au MLP

Le perceptron de Rosenblatt (1958) ne peut résoudre que des problèmes linéairement séparables (limite XOR, Minsky & Papert, 1969). Un Multi-Layer Perceptron (MLP) avec couches cachées surmonte cette limitation :

$$h^{(\ell)} = f(W^{(\ell)}h^{(\ell-1)} + b^{(\ell)})$$

où f est une activation non-linéaire. La sortie finale utilise Softmax pour la classification multi-classes.

3.2 Backpropagation

L'algorithme de rétropropagation (Rumelhart, Hinton & Williams, 1986) calcule le gradient par la règle de la chaîne, de la sortie vers l'entrée :

$$\delta^{(\ell)} = (W^{(\ell+1)})^\top \delta^{(\ell+1)} \odot f'(z^{(\ell)})$$

$$\frac{\partial L}{\partial W^{(\ell)}} = \delta^{(\ell)} (h^{(\ell-1)})^\top \quad W^{(\ell)} \leftarrow W^{(\ell)} - \eta \frac{\partial L}{\partial W^{(\ell)}}$$

Notre implémentation from scratch en NumPy atteint 0,952 d'accuracy sur Digits après 100 époques de mini-batch SGD.

3.3 Fonctions d'activation

Fonction	Formule	Dérivée	Usage
Sigmoid	$\sigma(x) = 1/(1+e^{-x})$	$\sigma(1-\sigma)$	Sortie binaire
Tanh	$\tanh(x)$	$1 - \tanh^2(x)$	LSTM, RNN
ReLU	$\max(0, x)$	1 si $x > 0$	Couches cachées
GELU	$x \cdot \Phi(x)$	—	Transformers

ReLU atténue le vanishing gradient : sa dérivée vaut 1 pour $x > 0$, contrairement à Sigmoid/Tanh (dérivée bornée par 0,25).

3.4 Optimiseurs — Adam

Adam (Kingma & Ba, 2015) adapte le taux d'apprentissage par paramètre :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \varepsilon}} \quad (\beta_1 = 0,9, \beta_2 = 0,999)$$

4. Pont ML ↔ DL : Dropout = Bagging Implicite

Gal & Ghahramani (ICML 2016) ont démontré que le Dropout (Srivastava et al., 2014) entraîne implicitement un ensemble de sous-réseaux :

Critère	Bagging (explicite)	Dropout (implicite)
Nb. sous-modèles	T modèles indépendants	2^M sous-réseaux
Poids	Indépendants	Partagés
Entraînement	Parallèle	Un masque par batch
Inférence	Vote / Moyenne	Poids $\times (1-p)$
Coût mémoire	$O(T)$	$O(1)$

Cette équivalence motive le MC-Dropout pour estimer l'incertitude et inspire directement notre pipeline hybride (rapport projet intégrateur).

5. Conclusion

Ce rapport a présenté les méthodes d'ensemble et les fondements des réseaux profonds en mettant en évidence leur complémentarité. Le lien Dropout = Bagging illustre que les principes mathématiques — réduction de variance par agrégation — transcendent la frontière ML/DL et motivent notre pipeline hybride sur HAM10000.

Bibliographie

- [1] Géron, A. (2023). *Hands-On Machine Learning* (3e éd.). O'Reilly.
- [2] Goodfellow, I., Bengio, Y. & Courville, A. (2016). *Deep Learning*. MIT Press.
- [3] Dietterich, T. G. (2000). Ensemble Methods in Machine Learning. *MCS 2000*.
- [4] Rumelhart, D. E., Hinton, G. E. & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.
- [5] Srivastava, N. et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *JMLR*, 15, 1929–1958.
- [6] Gal, Y. & Ghahramani, Z. (2016). Dropout as a Bayesian Approximation. *ICML 2016*.
- [7] Kingma, D. P. & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR 2015*.
- [8] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.

Transparence — Outils d'Intelligence Artificielle Utilisés

Conformément aux exigences du devoir, cette section documente de manière transparente l'ensemble des outils d'IA et des outils techniques utilisés, en précisant le rôle exact de chacun.

Outils d'Intelligence Artificielle

Outil	Usage dans le projet
DeepSeek	Planification initiale du projet ; structuration du prompt maître décrivant les livrables.
Claude (Anthropic) via Cowork	Génération HTML, notebooks Jupyter, rapports PDF, débogage.
GitHub Copilot	Assistance à l'installation des packages Python et débogage des erreurs de compatibilité.

Outils Techniques

Outil / Technologie	Rôle
HTML / CSS / JS (Vanilla)	Présentation interactive (52 slides, navigation clavier/souris, menu latéral).
KaTeX	Rendu des équations mathématiques LaTeX dans le navigateur.
Chart.js 4.4	Graphiques interactifs (Condorcet, courbes d'apprentissage).
ReportLab (Python)	Génération programmatique des rapports PDF avec en-tête/pied UN-FST.
VS Code / Jupyter / Colab	Environnement de développement et exécution des notebooks.
GitHub	Versionnement et partage du code source.
scikit-learn / XGBoost / PyTorch / SHAP	Implémentation des algorithmes ML/DL, interprétabilité et incertitude.

Déclaration d'intégrité académique

Tous les contenus scientifiques (théories, démonstrations mathématiques, pseudocodes, analyses de résultats) ont été vérifiés, compris et validés par les membres du groupe avant intégration dans les livrables. Les outils d'IA ont servi de levier de productivité (mise en forme, génération de code boilerplate, structuration documentaire), non de substitut à la compréhension ou à la réflexion scientifique.